

Why is SRL so tricky?

A CheckList for Tackling the Challenges of Semantic Role Labeling

Melina Paxinou

m.paxinou@student.vu.nl

Student ID: 2854344

Abstract

This paper evaluates SRL models trained on the Universal Proposition Bank dataset V1 (Propositions, 2020) using BERT and Logistic Regression. The methodology includes a challenge dataset inspired by the CheckList method by Ribeiro et al. (2020). The analysis shows how these models handle various syntactic structures and their performance across different roles. Results shed light on the strengths and weaknesses of both models, and provide insight on the models' capabilities in Semantic Role Labeling and their ability to adapt to diverse linguistic patterns.

1 Introduction

As traditional evaluation methods are based solely on held-out accuracy, they can result in an overestimation of a model's real-world performance. This is largely because these evaluations may not capture the model's behavior in a variety of linguistic contexts, leading to false confidence in their capabilities. If certain linguistic patterns are overrepresented in training data, while others are underrepresented or missing, the model may appear to perform well but fail in unseen contexts. This imbalance introduces bias, which impacts generalization and robustness.

To address this gap, behavioral testing has been proposed as an alternative evaluation framework. Rather than assessing only overall performance, behavioral testing is an evaluation approach that attempts to tackle this issue of biased outputs and focuses on the observable behaviors of a system, assessing how it performs under various inputs rather than examining its internal structure. In their paper *Beyond Accuracy: Behavioral Testing of NLP Models with CheckList*, Ribeiro et al. (2020) introduce CheckList, a structured testing methodology that systematically evaluates different aspects of the behavior of an NLP model. The methodology takes into consideration how different natural language

capabilities can be interpreted by a model and uses them to design targeted tests tailored to the specific task being evaluated.

CheckList is demonstrated using three NLP tasks; duplicate question detection, machine comprehension, and sentiment analysis. Duplicate question detection refers to the identification of whether a model can recognize when two different questions convey the same meaning. Using the CheckList method, the tests assessed whether the model could recognize semantic similarities despite variations in phrasing. Machine comprehension involves evaluating a model's ability to read and understand a given text and then answer questions based on its content. The evaluations focused on tests such as Vocab+POS and taxonomy, which helped identify the gaps in the models' comprehension abilities.

Sentiment analysis involves determining the emotion in a given text and classifying it as positive, negative, or neutral. The authors used CheckList to evaluate commercial Sentiment analysis models, including Microsoft's Text Analytics, Google Cloud's Natural Language, and Amazon's Comprehend, as well as models such as BERT-base and RoBERTa-base fine-tuned on the SST-2 dataset. Tests were created to target specific linguistic phenomena relevant to sentiment analysis. For example, tests assessed how well the models handled negation (e.g., determining sentiment from sentences like "not good"), whether they could maintain sentiment consistency during minor changes (such as changing adjectives), and how they responded to modified intensities (e.g., distinguishing between "very happy" and "happy"). The evaluation showed that, while the models had high performance results when using traditional evaluation methods, the CheckList method revealed a variety of bugs. These included the models' inability to manage basic linguistic phenomena associated with sentiment, such as failing to recognize sentiments around negated phrases.

The CheckList approach is evaluated through user studies that follow a series of steps, including how effectively users can identify bugs and test their models using the CheckList methodology, as well as comparisons with traditional testing. Participants provide feedback based on their experiences with CheckList, such as with reports on how the methodology helped with capability identification. Lastly, CheckList has also been applied in real-world contexts, such as with a team responsible for a commercial sentiment analysis model, where the team found bugs even in their extensively tested system. The implementation of CheckList in their testing processes revealed new insights, further demonstrating its practical utility.

Overall, the evaluation of the CheckList approach shows that it effectively uncovers bugs and improves the testing process for NLP models, providing insights beyond what traditional benchmarking offers.

The CheckList matrix is a tool that is used to evaluate NLP models by organizing the capabilities and test types. In this matrix, the rows represent the capabilities (i.e. Vocabulary, Negation, Named Entity Recognition), and the columns represent different test types, such as invariance and directional expectation tests that help evaluate how well a model performs on specific linguistic phenomena. Although it is beneficial to fill many cells with specific tests to gain insights on whether the models are able to generalize and perform well enough on data that are not similar to the dataset used, it is not necessary or possible to fill all cells. The tests that will ultimately be used depend on the task at hand, the model’s application, and context (e.g., what language is used in the data or what type of errors we are most concerned about). The general goal is to focus on areas where potential vulnerabilities or weaknesses are anticipated.

To implement CheckList, test instances are generated using templates that fill in specific linguistic constructs. At first, a small number of test instances are needed to adequately represent the linguistic phenomena that should be tested. Additionally, masked language models (MLMs) can play a significant role in this automatic generation process by providing predictions for the masked tokens within these templates, allowing for the creation of diverse and contextually relevant test cases at a faster pace.

The different test types used are Minimum Functionality Tests (MFTs), Invariance Tests (INV), and

Directional Expectation Tests (DIR). MFTs test whether the model can handle basic inputs, such as in sentiment analysis tests, ‘You are lame.’ can be used to determine whether the model can identify the negative sentiment. The test instances are straightforward, and the model is expected to perform relatively well. INV tests evaluate whether the model has a consistent output when faced with changes that should not logically affect it. In the same sentence, the model should produce the same results if ‘You’ was changed to ‘They’. Lastly, DIR tests assess how the model’s predictions vary when we implement controlled modifications to the input. For example, ‘You are lame.’ and ‘I dread you.’ both express negative sentiments; however, one is more negative than the other. The goal is to see whether these changes affect the model’s output, even though both should still be classified as negative.

2 Background

To apply the CheckList methodology to a Semantic Role Labeling (SRL) system, we first have to examine suitable capabilities and tests for this task. Having the capabilities suggested by [Ribeiro et al. \(2020\)](#) as a starting point, we can identify some that could be useful in an SRL system. Firstly, the Vocabulary+POS capability could be useful to test whether a model has learned the necessary vocabulary from the training set, as well as whether it is able to handle how words with different parts of speech may correspond to certain SRL arguments. Taxonomy, with the integration of synonyms and antonyms in places of arguments that the model handles correctly, can also be used in combination with core SRL arguments. Logic as a capability is also useful as it helps clarify whether a model can handle symmetry, consistency, and conjunctions, which, in turn, is crucial to understand how predicates relate to their arguments.

Furthermore, Negation may also be used, as understanding negation is a part of SRL, however, it is not a core argument of this task and may not be as significant depending on where the system is used. Robustness (e.g., with spelling mistakes) can also be a general NLP capability because NLP systems should be able to handle this type of variation and generalize appropriately. That being said, it is not necessarily a task-specific capability and should only be used to determine a more general function of a model.

On the other hand, Coreference as a capability would be irrelevant to an SRL system, as SRL focuses on identifying argument roles found in the same sentence and occurs around a single predicate, whereas Coreference as a task expands further than the limitations of a sentence. Temporal can also be considered as not useful, as the order in which events occur is irrelevant to SRL.

After testing the capabilities, the metric 'failure rate' is used. This metric measures the proportion of test cases in which the model performs incorrectly, that is, incorrectly predicts a label for the argument tested. However, when multiple tests are grouped together, such as in invariance tests, the failure rate reflects the combined percentage of failures across all related tests within that capability.

The capabilities suggested in this paper are a good starting point for creating tests that challenge NLP systems. However, existing linguistic resources such as PropBank, VerbNet, FrameNet, and WordNet can be valuable for designing capability tests specifically for an SRL system. Specifically, PropBank offers information on the arguments used with each verb and provides examples, showcasing some syntactic and lexical structures used with those arguments. By going through certain categories of verbs, we can see that they are matched with specific arguments, and therefore create capabilities that adequately challenge an SRL system. This process can be combined with the use of VerbNet, which provides verb classes categorized by sense that can be used to group verbs into capabilities. FrameNet can also be used to better understand the semantic roles and define conceptual frames and role relationships, while WordNet can help assess synonym and hyponym handling and generalization across lexical variations.

3 Challenging SRL

In Table 1, six different capabilities and twelve tests are proposed to assess whether SRL systems can handle some more linguistically challenging cases.

Dative Alternation for ARG1

This capability includes two minimal function tests (MFTs), each evaluating a different syntactic structure: one with a prepositional phrase and one with an indirect object. The tests aim to understand whether the models consistently identify ARG1, which in both cases is a direct object, even though the overall argument order changes. Since

the sentences used follow two distinct syntactic structures, an invariance (INV) test is conducted to compare how these structures affect the models' classification ability.

Dative Alternation for ARG2

This capability follows the same structure as the ARG1 alternation capability, using the same set of tests and test sentences but focuses on ARG2. ARG2 can appear as an indirect object or within a prepositional phrase, introducing syntactic variation. This test assesses how these structural changes affect the models' predictions for ARG2, especially for the Logistic Regression model, which relies on syntactic information to assign roles. As with ARG1, an INV test is included to compare model behavior across both sentence types.

Causative-Inchoative Alternation for ARG1

This capability consists of two MFTs, one for each type of verb: causative and inchoative verbs. The focus remains on the same argument, even though its syntactic role changes. For example, in the sentences *The sun melts the butter* (causative) and *The butter melts* (inchoative), *the butter* functions as an object in one case and as a subject in the other. Despite this syntactic shift, its semantic role does not change, which can confuse the models. An INV test is included to evaluate the consistency across these variations.

Involuntary Experience/Change Intransitive Verbs for ARG1

This capability includes a single MFT that focuses on ARG1 in sentences with involuntary experience or change-of-state verbs (e.g., *He died peacefully in his sleep after a long illness*). The challenge in this case occurs because ARG1 is more often the direct object, whereas in this case it functions as the subject.

Movement/Stative Verbs

This capability examines ARG1 in sentences containing movement or stative verbs and consists of a single MFT. In both cases, ARG1 indicates location or goal, which can easily be confused with other arguments or adjuncts that express these roles (e.g., ARG-M-LOC or ARG4). The test assesses whether the models correctly identify ARG1 in these contexts.

Robustness (Predicate Spelling Errors) for ARG1

This capability consists of test sentences that have been used for all previous capabilities and

have been successful. If taken as a separate test, it can be seen as an MFT with spelling errors applied to the predicate of each sentence; however, considering that the sentences have already been used and classified correctly, this test can also be seen as an INV test. Its goal is to determine whether minor spelling variations affect the predictions of the models for ARG1.

4 Creating a Challenge Dataset

The capabilities and tests described in Table 1 have been implemented and consist mainly of handcrafted examples. They have been written in a json file, which, for each test sentence, contains: the capability name, the test type, the string of the sentence, the tokens of the sentence, a list of binary values to identify the predicate, a list of gold labels (only for the considered argument for each test), the sentence number, the checked argument for reference purposes, and whether the test is used in an INV test.

Resources such as PropBank and VerbNet have been used in the making of the challenge dataset, both for inspiration and for directions on how to group verbs with similar properties together. Specifically, in the *Movement/Stative Verbs for ARG1* capability, all test examples have been copied directly from PropBank (PropBank, 2020). In all other cases, we handcrafted the examples both with inspiration from PropBank and with ideas taken from ChatGPT, which were filtered and changed to fit the PropBank arguments.

While ChatGPT was used for inspiration as well, it did not provide us with good enough examples with correct corresponding arguments, therefore, all the examples generated automatically have been filtered and changed to fit the test criteria. ChatGPT does not understand the concepts of predicates and arguments, so the prompts that mentioned roles did not provide us with the correct output and could not be used as they were. Even in cases where we provided it with specific verb classes and argument structures as examples, it did not manage to give us verbs that had the corresponding checked arguments for the tests of our choice. One case where a semiautomatic generation method was used is this [ChatGPT output](#) to create test instances for the *Involuntary Experience/Change Intransitive Verbs for ARG1* capability. As shown in the output, ChatGPT initially produced examples that contained ARG1 but not as a subject. After providing it with

the correct examples, it produced more relevant test instances, which still required manual PropBank verification.

It should be noted that the Challenge dataset is somewhat imbalanced. The first two capabilities (*Dative Alternation for ARG1* and *Dative Alternation for ARG2*) contain the most instances among the capabilities that test the core arguments of SRL, and *Robustness* contains the largest number of test examples out of all the tests in the dataset. The *Movement/Stative Verbs for ARG1* capability contains the least number of examples, as it was the most difficult to create and required the most amount of PropBank verification. ARG1 is the argument that is tested most frequently, as it is the focus of most tests, and ARG2 was tested in a single capability.

Lastly, the examples used mainly focus on specific linguistic challenges rather than more general or simple cases. They help explore less common structures and were chosen with respect to the complexity of human language. An SRL system should be able to correctly classify cases similar to the ones we used in the dataset, as they reflect core SRL arguments that carry a great deal of information.

5 Model Descriptions

5.1 Logistic Regression

Logistic Regression is a binary classification algorithm that handles discrete data and predicts the probability of a binary outcome. It is a probabilistic classifier and requires a feature representation of the input data which, in the case of SRL, can be translated into syntactic dependencies or information about named entities. Its architecture also requires a classification function (sigmoid), a cross-entropy loss function that calculates loss, and a gradient descent algorithm for optimizing the loss function. However, if the task at hand is a multi-class classification problem, other types of classification can be used, such as Multinomial Logistic Regression (Jurafsky and Martin, 2024).

When using Logistic Regression for SRL, the creation of meaningful features specific for this task is necessary. The first feature used is the directed dependency path from a given token to the predicate and the lemma of the predicate, with added arrows to show parent-child relations ($\uparrow$ child-parent and $\downarrow$ parent-child) and explicit dependency labels. By adding directions, the feature differentiates be-

Capability	Test Type & Description	Example Test Cases	Expected Behaviour
Dative Alternation for ARG1	MFT – short sentences with prepositional phrase and direct object	She gives[PRED] a book to her friend.	book = ARG1
	MFT - short sentences with indirect object (dative) and direct object	She gives[PRED] her friend a book.	book = ARG1
	INV – combination of both test sentences	She gives[PRED] a book to her friend. She gives[PRED] her friend a book.	book = ARG1
Dative Alternation for ARG2	MFT – short sentences with prepositional phrase and direct object	She gives[PRED] a book to her friend.	friend = ARG2
	MFT – short sentences with indirect object (dative) and direct object	She gives[PRED] her friend a book.	friend = ARG2
	INV – combination of both test sentences	She gives[PRED] a book to her friend. She gives[PRED] her friend a book.	friend = ARG2
Causative-Inchoative Alternation for ARG1	MFT – short sentences with causative verbs	The sun melts[PRED] the butter.	butter = ARG1
	MFT – short sentences with inchoative verbs	The butter melts[PRED].	butter = ARG1
	INV – combination of both test sentences	The sun melts[PRED] the butter. The butter melts[PRED].	butter = ARG1
Involuntary Experience/Change Intransitive Verbs for ARG1	MFT – medium complexity sentences with involuntary experience or change intransitive verbs	He died[PRED] peacefully in his sleep after a long illness.	He = ARG1
Movement/Stative Verbs for ARG1	MFT – high complexity sentences with movement and stative verbs and substantives	During the time of his residence[PRED] in Mourhead, Michael Higgins was united in marriage to Agnes Peterson.	Mourhead = ARG1
Predicate Spelling Errors (Robustness) for ARG1	INV – sentences with typos on the predicate, from the above tests that have been correctly predicted	She gies[PRED] a book to her friend.	book = ARG1

Table 1: Proposed Capabilities and Tests for SRL System Evaluation

tween arguments that control the predicate (e.g., subjects) and those dependent on it (e.g., objects, adjuncts), while the predicate’s dependency label is intentionally omitted from the path. The lemma of the predicate is added at the end of the path instead of the dependency label to avoid longer paths that would conclude in the same word (e.g., ‘ROOT’). Including the predicate’s lemma allows the model to generalize over different inflected forms, allowing faster computation and efficiency, while ensuring that the focus remains on the core lexical semantics rather than surface variations.

The second feature used contains information about NER labels and words that precede named entities or nouns. Named Entity labels are important for SRL, as they often play specific roles in a sentence, such as agents or patients in a predicate’s argument structure. For example, the phrase ‘to Amsterdam’ conveys a locative meaning different from simply ‘Amsterdam’, and explicitly encoding such information helps the model distinguish between different argument roles. Furthermore, in cases where NER labels are not applicable, part-of-speech information is provided. Since named entities are relatively sparse, relying solely on them would often result in many tokens being labeled as O. This combined feature, however, is created so that the model can generalize better.

The last feature used is the constituency path from a given word to the predicate combined with the label ‘_PREDICATE’ to mark the position of the predicate in question. In Semantic Role Labeling, understanding the syntactic structure of a sentence is crucial to accurately identify the role of an argument in relation to the predicate. A constituency path provides a representation of the syntactic relationship between a token and the predicate by capturing the hierarchical path in a constituency parse tree. These paths provide general information about the sentence and help differentiate between core arguments and adjuncts based on their positions within the tree. By marking the predicate explicitly we can show the model where the predicate is in the constituency tree, and by truncating the path for words that come after the predicate, we ensure efficiency and speed in our model.

The Logistic Regression classification report (Figure 1) reveals fairly good model performance, with precision consistently higher than recall. The model’s high weighted average performance is largely due to the dominance of the O label, which makes up over 90% of the dataset and achieves near-perfect precision (0.954) and recall (0.990). However, this does not reflect the model’s ability to distinguish semantic roles. Among the core argu-

ments, ARG1 and ARG0 perform moderately well, with F1-scores of 0.665 and 0.624, although both have low recall (0.583 and 0.534). ARG2 performs slightly worse (F1 = 0.641, recall = 0.580). Adjunct roles are harder to classify and have relatively low results, with the exception of ARGM-MOD and ARGM-NEG, which perform relatively well (F1 = 0.748, 0.642). In general, the model is reluctant to assign positive labels, leading to low recall in most roles. Less frequent labels (fewer than 100 occurrences) perform poorly, often with F1-scores of 0.

The expected performance of Logistic Regression on the Challenge Dataset heavily relies on the syntactic structures and features it has encountered during training. It is expected to perform well in the first two capabilities, as they involve more regular and consistent structures that the model is more likely to have seen before. For causative-inchoative alternations, the model will likely perform better on causative structures. This is because the checked argument in this case (ARG1) typically corresponds to a direct object, which is more frequent and easier to identify. However, in inchoative structures, ARG1 functions as the syntactic subject, making it less predictable. A similar pattern is expected for involuntary experience/change of state verbs, where the argument behaves similarly to inchoative constructions. For movement/stative verbs, performance is more likely to drop due to the increased complexity of the sentences and the confusion between ARG1 that expresses location or goal and ARG4 or adjuncts (e.g., ARGM-LOC). Regarding robustness, performance is expected to drop moderately, as logistic regression relies on a feature that uses the lemma of the predicate. However, since the other selected features are based on predicate positions and syntactic relations, they should still be able to extract dependency or constituency paths, NER labels, and POS tags, which could provide enough information for reasonable predictions.

5.2 BERT

Bidirectional Encoder Representations from Transformers (BERT) is an NLP model that processes language bidirectionally, taking into account both the left and the right context of each word in a sentence. This approach allows BERT to understand the dependencies, discern nuances, and recognize relationships between words, thus analyzing at a depth that other types of model cannot achieve

(Wang, 2024). BERT can be fine-tuned to support the recognition of how predicates interact with surrounding tokens, which is what makes it highly suitable for this task. Its subword tokenization ensures that all words influenced by a given predicate will be handled effectively by separating them into subtokens that do not necessarily match the input tokens, i.e. the existing words in the dataset that have already been tokenized.

BERT also provides us with the option to add a special token to its vocabulary: [PRED]. The special token is responsible for directing its attention to the important part of the sentence, that is, the predicate. The special negation token is added before each predicate occurrence, so that it signals its position. This augmentation technique helps BERT to better contextualize the predicates and their surrounding tokens.

BERT requires specific input to train and classify; therefore, a number of preprocessing steps have been added. First, each sentence is converted into a list of dictionaries that contain the following keys: tokens, labels, and predicate. Special tokens are added to each dictionary ([PRED]) to mark the predicate with the augmentation technique used in 'NegBERT: A Transfer Learning Approach for Negation Detection and Scope Resolution' by Khandelwal and Sawant (2020). As special tokens are added, the rest of the dictionary keys are updated with the appropriate padding. Then, the input is given to the tokenizer, which splits the words into subtokens. The input now contains `input_ids`, `attention_mask`, `word_ids`, and `tokenized_tokens`, and is passed to the final preprocessing step, which utilizes the Datasets library and only retains the keys `input_ids`, `attention_mask`, and `labels`. This step ensures that the input is in a form recognizable by BERT.

After the predictions have been made, they are added to the dictionary along with the gold labels to be able to create the classification report and confusion matrix. The number of subtokens for each sentence is calculated by counting the word ids, as the remaining predictions and labels are padding. Then, the word mappings are used to return the subtokens in their original form and ensure that the majority of predictions is kept for each token. If there is a tie, then the first token's prediction is chosen.

BERT's predictions have proven to be better compared to Logistic Regression, as seen in the

classification report (Figure 2), but its high accuracy is largely due to the dominance of the O label, similarly to the previous model evaluation. Core arguments such as ARG0 and ARG1 are identified with high precision and recall, achieving F1-scores of 0.874 and 0.875, respectively. ARG2 performs slightly worse (F1-score: 0.778) but is still well detected. However, rarer roles such as ARG3 are almost entirely missed, with F1-scores close to 0 due to their low frequency. Similarly, some ARGM labels are difficult to classify, but ARGM-ADJ, ARGM-ADV, and ARGM-TMP achieve moderate to high success (F1-scores: 0.744, 0.677, and 0.840). Generally, BERT is very successful at identifying frequent roles, but struggles with underrepresented ones, leading to poor performance on rare labels.

BERT is expected to perform better compared to Logistic Regression on the Challenge Dataset, particularly in the first three capabilities, across all relevant tests. Given that these sentences follow a relatively simple syntactic structure, the model should have little difficulty understanding the context and making accurate predictions. For the last two core capabilities, performance is more likely to worsen due to the increased syntactic complexity. However, it should still perform better than Logistic Regression, as BERT’s deep contextual representations allow it to handle more complex sentence structures. Regarding robustness, while perfect performance is not expected, the number of misclassifications should remain relatively stable and not be significantly affected by spelling errors.

6 Results

The results presented in Table 2 indicate several important observations regarding the performance of the models.

The results on the *Dative Alternation for ARG1* tests show relatively low failure rates, especially for BERT, with failure rates of 7.69% for the ARG1-Prepositional Phrase test and 0.00% for the ARG1-Dative test. Logistic Regression performs slightly worse, but overall its performance is fairly accurate. The invariance test shows that Logistic Regression has a relatively higher failure rate of 30.77%, whereas BERT maintains a low error rate of 7.69%, yet the overall results are quite good.

Similarly, the results for the Dative Alternation for ARG2 are promising, with BERT achieving perfect performance on the ARG2-Prepositional

Phrase test (0% failure rate) and a slightly higher failure rate (7.69%) on the ARG2-Dative test. Logistic Regression, on the other hand, misclassifies almost half of the test instances on the ARG2-Prepositional Phrase test (46.15%) and more than half on the ARG2-Dative test (69.23%). As seen in the INV test, Logistic Regression struggles significantly with an 84.62% failure rate, while BERT once again performs very well with a failure rate of only 7.69%.

In the case of the Causative-Inchoative Alternation for ARG1, we observe an even larger gap between BERT and Logistic Regression. BERT manages to classify correctly all test instances, while Logistic Regression reaches a failure rate of 77.78% at the Inchoative-ARG1 test. In the INV test, we can observe that Logistic Regression exhibits a high failure rate of 77.78%, while BERT correctly classifies all instances.

Looking at the capabilities that contain only one MFT each, we can observe that the results for the Movement/Stative Verbs for ARG1 test show a significant difference between the models, with BERT having a notably higher failure rate (87.50%) compared to LogReg (62.50%). For the Involuntary Experience/Change Intransitive Verbs for ARG1 test, the failure rates are more balanced, with LogReg having a failure rate of 50% and BERT achieving 30%.

Lastly, the Predicate Spelling Errors (Robustness) for ARG1 invariance test shows a stark contrast, with Logistic Regression failing 93.10% of the time, whereas BERT remains robust with only a 6.90% failure rate.

7 Discussion

The results described above reveal several intriguing patterns, which highlight the differences in how BERT and Logistic Regression handle the linguistic challenges posed through the creation of this dataset. It should be noted that for the analysis of these results, the small sample of test sentences used must be considered in terms of the limitations this Challenge Dataset has and its ability to generalize. Given a larger dataset, the failure rates and respective conclusions presented in this paper might change.

The relatively good results in both Logistic Regression and BERT on the *Dative Alternation for ARG1* tests most likely relate to the fact that ARG1 is a core argument of SRL and has the syntactic role

Capability	Checked Argument	Num Instances	Num LogReg Incorrect	Num BERT Incorrect	LogReg Failure Rate (%)	BERT Failure Rate (%)
Dative Alternation for ARG1	ARG1 and PP	13	3	1	23.08	7.69
Dative Alternation for ARG1	ARG1 and Dative	13	4	0	30.77	0.00
Dative Alternation for ARG1	INVARIANCE	–	–	–	30.77	7.69
Dative Alternation for ARG2	ARG2-PP	13	6	0	46.15	0.00
Dative Alternation for ARG2	ARG2-Dative	13	9	1	69.23	7.69
Dative Alternation for ARG2	INVARIANCE	–	–	–	84.62	7.69
Causative-Inchoative Alternation for ARG1	Causative-ARG1	9	5	0	55.56	0.00
Causative-Inchoative Alternation for ARG1	Inchoative-ARG1	9	7	0	77.78	0.00
Causative-Inchoative Alternation for ARG1	INVARIANCE	–	–	–	77.78	0.0
Movement/Stative Verbs for ARG1	ARG1	8	5	7	62.50	87.50
Involuntary Experience/Change Intransitive Verbs for ARG1	ARG1	10	5	3	50.00	30.00
Predicate Spelling Errors (Robustness) for ARG1	INVARIANCE-ARG1	29	27	2	93.10	6.90

Table 2: Comparison of Logistic Regression and BERT Failure Rates

of direct object in all test sentences. This allows the features of Logistic Regression to be utilized to the maximum, and does not pose challenges on BERT.

On the contrary, the tests for *Dative Alternation for ARG2* are not that successful for Logistic Regression. This may have to do with the fact that Logistic Regression relies on the syntactic relations between the predicate and the given tokens and, in constructions where the most common roles are described by different syntactic structures than the ones that Logistic Regression has mostly been trained on, it is more difficult to produce correct classifications. BERT, however, is highly capable of distinguishing the role of ARG2 in both types of construction.

The result of the tests for *Causative-Inchoative Alternation for ARG1* suggest that the system faces particular challenges with constructions that include inchoative verbs, most likely due to the features used during training. Logistic Regression has access to the syntactic information of each token and is most likely confused by the fact that the tokens in question have the role of syntactic subject, yet are still ARG1. BERT performs perfectly on this test, suggesting that it is able to capture the semantic role of ARG1 in both causative and inchoative constructions to a great extent.

The results for *Movement/Stative Verbs for ARG1* show a significant difference in model performance, with BERT performing much worse than Logistic Regression. This could suggest that Lo-

gistic Regression’s features focusing mostly on the syntactic aspects of the training set actually help the model differentiate between the sense of location or goal and the syntactic characteristics of ARG1. On the other hand, BERT has access to a variety of lexical information, which may lead to it considering these test examples as locations or goals.

The *Involuntary Experience/Change Intransitive Verbs for ARG1* test shows a notable performance gap between the two models, with Logistic Regression performing worse. This, again, may have to do with the fact that BERT is able to take into consideration both syntactic and semantic information, while Logistic Regression’s features focus on syntax. In this case, similarly to the *Causative-Inchoative Alternation for ARG1* test, ARG1 has the syntactic role of a subject and is likely the reason why Logistic Regression is not as effective.

With 29 instances, the *Predicate Spelling Errors (Robustness) for ARG1* test is better represented, and the performance is more likely to reflect the models’ true capabilities in handling noisy data. Specifically, the interesting thing about this case is that Logistic Regression performs extremely badly on this test, yet BERT is not affected by spelling errors at all. This shows that BERT is able to handle noisy data extremely well, while Logistic Regression is not.

In conclusion, while the results provide insight into the strengths and weaknesses of the models, particularly in identifying areas of difficulty, the

small number of test instances significantly limits the reliability of the findings. The results cannot be fully generalized to a larger dataset, and further testing with larger, more diverse datasets is likely to have provided different results. Overall, the performance variability across different test types highlights the need for targeted evaluation approaches, such as the CheckList method, especially in complex tasks like SRL.

8 Future Work: SRL for doctor-patient conversations during a GP appointment

As seen previously in this paper, SRL plays a critical role in understanding the syntactic and semantic relationships in a given text. It answers questions such as 'who', 'what', and 'whom', and helps with mapping information in several applications, such as question answering, information extraction, and summarization. By capturing the structure of events and participants, SRL significantly improves a system's ability to interpret and process language in domain-specific tasks.

However, applying SRL to specialized domains is a challenging task, as labeled data for the specific domain may be insufficient. The language structure, vocabulary, and patterns change in domain-specific data, making the adaptation of SRL and other NLP tasks fairly difficult when it comes to adapting them to the target domain. In this paper, the focus will be on training and evaluating an SRL system with access to an unlimited amount of data from various domains, a small corpus of labeled doctor-patient conversations, and a larger corpus of unlabeled doctor-patient conversations.

Initially, the SRL model was trained on the unlimited Universal Proposition Bank data from various domains using a pre-trained BERT. This model will be applied to the small labeled target-domain dataset without any domain-specific adaptation to establish a baseline performance. This step will allow us to first check how much the model trained on generic data is able to capture on its own and provide a performance benchmark before any domain-specific fine-tuning or adaptation is introduced.

Considering that the available labeled data is limited, pseudo-labeling will be used to create pseudo-labels for unlabeled data containing doctor-patient conversations (Jia et al., 2019). Pseudo-labeling or self-training is a semi-supervised machine learning method that utilizes a small set of labeled data to train a basic model, which is then used to pre-

dict labels for unlabeled data, that is, pseudo-labels. This data is then added to the training set, and the model is trained on this larger version of the dataset containing the initial small labeled dataset and the larger dataset with the pseudo-labels (Dantas, 2023).

One downside of pseudo-labeling is the introduction of noise, as labels generated by the model's predictions can often be incorrect. This may lead to overfitting, where the neural network becomes too specialized in the noisy data. As a result, the generalization ability of the model to new, unseen data could be compromised, especially when the noise in the pseudo-labels is significant (Ye et al., 2020).

To evaluate, one possibility would be to set aside a small sample of the initial labeled domain-specific dataset. This small portion of the initial dataset will be used to calculate metrics such as F1-Score, Precision, and Recall, and compare to the baseline established with the generic SRL model.

As this model could be used in real-life situations where attention to detail is important, it would need to be thoroughly evaluated. For that reason, a challenge dataset would need to be created using the CheckList method (Ribeiro et al., 2020). Taking into account the purpose and use of this SRL model, the capabilities tested and templates created would require domain-specific information, such as medical terminology, abbreviations, syntactic structures common in healthcare contexts, and references to diseases or pharmaceuticals. In addition to the core capabilities of SRL, the challenge set would need to also cover locations, time, dates, and negation, as this is the kind of information that could define when and where a patient started experiencing symptoms, as well as to understand the context of diagnoses, treatments, or medical history. In addition, a qualitative error analysis would need to be performed to identify persistent failure cases and areas where the model still has difficulties despite the adaptation process.

Ultimately, SRL is a fundamental tool in medical NLP, as it enables systems to understand doctor-patient interactions by identifying semantic roles from conversations. The use of challenge sets is highly important in such sensitive applications as it ensures that the models used in real-life situations are thoroughly tested on particularly challenging data and are able to handle the domain-specific language use that accompanies medical conversa-

tions. Lastly, as labeled data are usually difficult to acquire in all applications and languages, pseudo-labeling and other unsupervised or semi-supervised machine learning methods are essential to create NLP models with limited resources.

9 Conclusion

In this paper, Semantic Role Labeling (SRL) models are tested on a challenge dataset inspired by the work of Ribeiro et al. (2020). Using models such as BERT and Logistic Regression, the impact of various syntactic phenomena on SRL performance is analyzed. The findings suggest that Logistic Regression heavily relies on the given features, primarily syntactic and named entity information, while BERT has a more well-rounded grasp of the task, even though it has been observed to fail in cases where the argument boundaries are unclear.

Acknowledgements

I would like to acknowledge Elisabetta Denticò for her contribution to feature selection, challenge dataset creation, and domain adaptation sections of this paper.

References

- Nielsen Castelo Damasceno Dantas. 2023. [Data augmentation in nlp](#). Accessed: 2025-03-28.
- Chen Jia, Xiaobo Liang, and Yue Zhang. 2019. [Cross-domain ner using cross-domain language modeling](#). In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 2464–2474. Association for Computational Linguistics.
- Daniel Jurafsky and James H. Martin. 2024. *Speech and Language Processing*. Draft version, January 12, 2025. Retrieved from <https://web.stanford.edu/~jurafsky/slp3/5.pdf>.
- Abhishek Khandelwal and Shrikant Sawant. 2020. [Negbert: A transfer learning approach for negation detection and scope resolution](#). In *Proceedings of the 12th Conference on Language Resources and Evaluation (LREC 2020)*, pages 5739–5748. European Language Resources Association (ELRA).
- PropBank. 2020. Propbank frames v3.4.0. <https://propbank.github.io/v3.4.0/frames/>. Accessed: 2025-03-28.
- Universal Propositions. 2020. Up-1.0: Universal proposition bank (english-ewt). <https://github.com/UniversalPropositions/UP-1.0>. Accessed: 2025-03-28.
- Marco Tulio Ribeiro, Tongshuang Wu, Carlos Guestrin, and Sameer Singh. 2020. [Beyond accuracy: Behavioral testing of nlp models with checklist](#). In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 4902–4912. Association for Computational Linguistics.
- Jo Wang. 2024. [Bert](#). Accessed: 2025-03-28.
- Hai Ye, Qingyu Tan, Ruidan He, Juntao Li, Hwee Tou Ng, and Lidong Bing. 2020. [Feature adaptation of pre-trained language models across languages and domains with robust self-training](#). In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 7386–7399. Association for Computational Linguistics.

Appendix

	precision	recall	f1-score	support
ARG0	0.751	0.534	0.624	1733
ARG1	0.773	0.583	0.665	3241
ARG1-DSP	0.000	0.000	0.000	4
ARG2	0.716	0.580	0.641	1129
ARG3	0.769	0.270	0.400	74
ARG4	0.424	0.500	0.459	56
ARG5	0.000	0.000	0.000	1
ARGA	0.000	0.000	0.000	2
ARGM-ADJ	0.750	0.395	0.517	228
ARGM-ADV	0.458	0.165	0.243	496
ARGM-CAU	0.000	0.000	0.000	46
ARGM-COM	1.000	0.077	0.143	13
ARGM-CXN	1.000	0.083	0.154	12
ARGM-DIR	0.188	0.064	0.095	47
ARGM-DIS	0.878	0.198	0.323	182
ARGM-EXT	0.750	0.400	0.522	105
ARGM-GOL	0.000	0.000	0.000	24
ARGM-LOC	0.630	0.164	0.261	207
ARGM-LVB	0.737	0.203	0.318	69
ARGM-MNR	0.333	0.068	0.112	148
ARGM-MOD	0.896	0.643	0.748	442
ARGM-NEG	0.842	0.519	0.642	216
ARGM-PRD	1.000	0.045	0.087	44
ARGM-PRP	0.762	0.213	0.333	75
ARGM-PRR	0.455	0.217	0.294	69
ARGM-TMP	0.498	0.254	0.337	543
C-ARG0	0.000	0.000	0.000	3
C-ARG1	0.875	0.269	0.412	52
C-ARG1-DSP	0.000	0.000	0.000	1
C-ARG2	0.000	0.000	0.000	7
C-ARG3	0.000	0.000	0.000	2
C-ARGM-CXN	0.000	0.000	0.000	5
C-ARGM-LOC	0.000	0.000	0.000	1
C-V	0.000	0.000	0.000	16
O	0.954	0.990	0.972	91718
R-ARG0	0.773	0.254	0.382	67
R-ARG1	0.625	0.192	0.294	52
R-ARG2	0.000	0.000	0.000	1
R-ARGM-ADV	0.000	0.000	0.000	1
R-ARGM-DIR	0.000	0.000	0.000	1
R-ARGM-LOC	0.000	0.000	0.000	9
R-ARGM-MNR	0.000	0.000	0.000	8
R-ARGM-TMP	0.000	0.000	0.000	2
accuracy			0.941	101152
macro avg	0.415	0.183	0.232	101152
weighted avg	0.932	0.941	0.933	101152

Figure 1: Logistic Regression Classification Report

Classification Report:				
	precision	recall	f1-score	support
ARG0	0.864	0.883	0.874	1733
ARG1	0.860	0.890	0.875	3241
ARG1-DSP	0.000	0.000	0.000	4
ARG2	0.770	0.787	0.778	1129
ARG3	0.333	0.014	0.026	74
ARG4	0.674	0.554	0.608	56
ARG5	0.000	0.000	0.000	1
ARGA	0.000	0.000	0.000	2
ARGM-ADJ	0.725	0.763	0.744	228
ARGM-ADV	0.721	0.637	0.677	496
ARGM-CAU	0.577	0.652	0.612	46
ARGM-COM	0.500	0.308	0.381	13
ARGM-CXN	0.571	0.333	0.421	12
ARGM-DIR	0.490	0.511	0.500	47
ARGM-DIS	0.777	0.786	0.781	182
ARGM-EXT	0.780	0.743	0.761	105
ARGM-GOL	1.000	0.042	0.080	24
ARGM-LOC	0.584	0.691	0.633	207
ARGM-LVB	0.722	0.754	0.738	69
ARGM-MNR	0.590	0.554	0.571	148
ARGM-MOD	0.966	0.964	0.965	442
ARGM-NEG	0.945	0.954	0.949	216
ARGM-PRD	0.571	0.273	0.369	44
ARGM-PRP	0.500	0.573	0.534	75
ARGM-PRR	0.649	0.696	0.671	69
ARGM-TMP	0.808	0.875	0.840	543
C-ARG0	0.000	0.000	0.000	3
C-ARG1	0.578	0.500	0.536	52
C-ARG1-DSP	0.000	0.000	0.000	1
C-ARG2	1.000	0.143	0.250	7
C-ARG3	0.000	0.000	0.000	2
C-ARGM-CXN	0.000	0.000	0.000	5
C-ARGM-LOC	0.000	0.000	0.000	1
C-V	0.714	0.625	0.667	16
O	0.992	0.991	0.992	91718
R-ARG0	0.870	0.896	0.882	67
R-ARG1	0.656	0.769	0.708	52
R-ARG2	0.000	0.000	0.000	1
R-ARGM-ADV	0.000	0.000	0.000	1
R-ARGM-DIR	0.000	0.000	0.000	1
R-ARGM-LOC	0.500	0.556	0.526	9
R-ARGM-MNR	0.000	0.000	0.000	8
R-ARGM-TMP	0.000	0.000	0.000	2
accuracy			0.975	101152
macro avg	0.495	0.435	0.441	101152
weighted avg	0.975	0.975	0.975	101152

Figure 2: BERT Classification Report